Gradient Boosting Classifier

高怡宣
陽明大學 生物醫學影像暨放射科學系

Gradient Boosting classification of the Iris dataset

Code midified from: Ryan Wang, Prediction with Gradient Boosting Classifier

https://www.kaggle.com/beagle01/prediction-with-gradient-boosting-classifier

In [1]:
```
import pandas as pd
iris = pd.read_csv("Iris.csv")
```

In [2]:
```
X = iris.drop(['Id','Species'],axis=1)
y = iris['Species']
```

In [4]:
```
# transform data to a given range. default=(0, 1)
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
X_scale = scaler.fit_transform(X)
```

[5]:
```
# split training feature and target sets into training and validation subsets
from sklearn.model_selection import train_test_split

X_train, X_validation, y_train, y_validation =
train_test_split(X_scale, y, random_state=0, test_size=0.2)
```

[6]:
```
print(X_train.shape)
print(y_train.shape)
print(X_validation.shape)
print(y_validation.shape)
```

```
(120, 4)
(120,)
(30, 4)
(30,)
```

```python
# import machine learning algorithms
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_curve, auc
```

```python
# train with Gradient Boosting algorithm
# compute the accuracy scores on train and validation sets when training with differen

learning_rates = [0.05, 0.1, 0.25, 0.5, 0.75, 1]
for learning_rate in learning_rates:
    gb = GradientBoostingClassifier(n_estimators=20, learning_rate = learning_rate, m
    gb.fit(X_train, y_train)
    print("Learning rate: ", learning_rate)
    print("Accuracy score (training): {0:.3f}".format(gb.score(X_train, y_train)))
    print("Accuracy score (validation): {0:.3f}".format(gb.score(X_validation, y_vali
    print()
```

```
Learning rate:  0.05
Accuracy score (training): 0.967
Accuracy score (validation): 1.000

Learning rate:  0.1
Accuracy score (training): 0.967
Accuracy score (validation): 1.000

Learning rate:  0.25
Accuracy score (training): 1.000
Accuracy score (validation): 1.000
```

```
Learning rate:  0.5
Accuracy score (training): 1.000
Accuracy score (validation): 1.000

Learning rate:  0.75
Accuracy score (training): 1.000
Accuracy score (validation): 1.000

Learning rate:  1
Accuracy score (training): 1.000
Accuracy score (validation): 1.000
```

3.2.4.3.5. sklearn.ensemble.GradientBoostingClassifier

class sklearn.ensemble. **GradientBoostingClassifier** (*loss='deviance', learning_rate=0.1, n_estimators=100, subsample=1.0, criterion='friedman_mse', min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_depth=3, min_impurity_decrease=0.0, min_impurity_split=None, init=None, random_state=None, max_features=None, verbose=0, max_leaf_nodes=None, warm_start=False, presort='auto', validation_fraction=0.1, n_iter_no_change=None, tol=0.0001*) [source]

Gradient Boosting for classification.

GB builds an additive model in a forward stage-wise fashion; it allows for the optimization of arbitrary differentiable loss functions. In each stage `n_classes_` regression trees are fit on the negative gradient of the binomial or multinomial deviance loss function. Binary classification is a special case where only a single regression tree is induced.

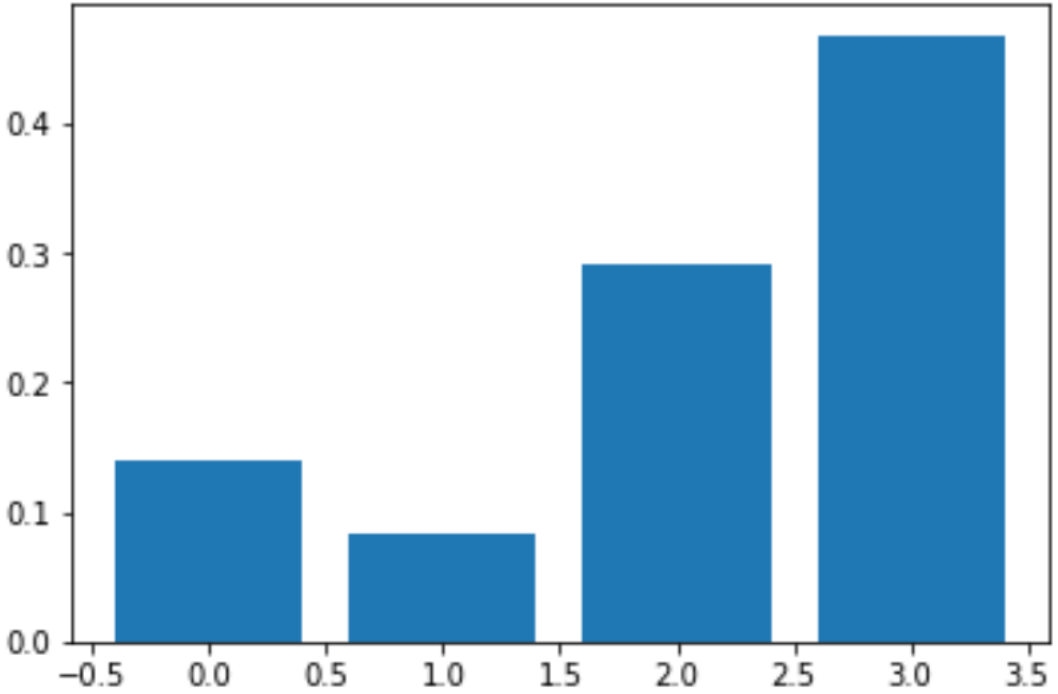
https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html

```
# Output confusion matrix and classification report of Gradient Boosting

gb = GradientBoostingClassifier(n_estimators=20, learning_rate = 0.5, max
gb.fit(X_train, y_train)
predictions = gb.predict(X_validation)

print("Confusion Matrix:")
print(confusion_matrix(y_validation, predictions))
print()
print("Classification Report")
print(classification_report(y_validation, predictions))
```

```
Confusion Matrix:
[[11  0  0]
 [ 0 13  0]
 [ 0  0  6]]

Classification Report
                 precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        11
Iris-versicolor       1.00      1.00      1.00        13
 Iris-virginica       1.00      1.00      1.00         6

    avg / total       1.00      1.00      1.00        30
```

```
from matplotlib import pyplot
print(gb.feature_importances_)
# plot
pyplot.bar(range(len(gb.feature_importances_)), gb.feature_importances_)
pyplot.show()
```

```
[0.14066605 0.08292343 0.29134209 0.46840176]
```



Thank you for your attention